

Segurança na Internet

sexta-feira, 10 de abril de 2026 10:40

Vulnerabilidades comuns em aplicações web

Para Agra (2019), a internet é utilizada para inúmeras atividades, como serviços bancários, compras, educação, comunicação, notícias, entretenimento, redes sociais, etc. Pelo fato de ter grande acesso e estar em constante ascensão, há uma preocupação com a segurança das informações e a privacidade na web. Vamos verificar algumas dessas vulnerabilidades:

- **Injeção de SQL (SQL Injection):** essa vulnerabilidade ocorre quando os invasores inserem comandos SQL maliciosos em campos de entrada da aplicação. Isso pode permitir que hackers acessem, modifiquem ou excluam dados do banco de dados.
- **Cross-Site Scripting (XSS):** atacantes inserem scripts maliciosos em páginas da web que são, então, executados nos navegadores dos usuários finais. Isso pode permitir o roubo de informações de sessão ou a exibição de conteúdo malicioso.
- **Cross-Site Request Forgery (CSRF):** nesse tipo de ataque, os invasores induzem os usuários a executar ações não intencionais em uma aplicação, explorando a confiança do navegador do usuário.
- **Autenticação fraca:** senhas fracas ou métodos de autenticação inseguros podem permitir que invasores adivinhem ou crackeiem credenciais de acesso.
- **Injeção de código no lado do servidor (Server-Side Code Injection):** ataques como a injeção de PHP permitem que os invasores executem código no lado do servidor, comprometendo o sistema.
- **Configurações incorretas de segurança:** erros de configuração, como permissões excessivas de arquivo ou falta de atualizações de segurança, podem abrir portas para invasores.
- **Integração de componentes não confiáveis:** usar componentes de terceiros sem verificar sua segurança pode introduzir vulnerabilidades na aplicação.
- **Controle de acesso insuficiente:** pode permitir que usuários não autorizados acessem funcionalidades restritas.
- **Vulnerabilidades de File Upload:** permitir que os usuários façam upload de arquivos sem verificar sua segurança pode resultar em execução de código malicioso.
- **Falhas de segurança na API:** aplicações web modernas frequentemente dependem de API. Vulnerabilidades em API podem expor dados sensíveis ou permitir ações não autorizadas.

Compreender e corrigir essas vulnerabilidades é fundamental para garantir a segurança de aplicações web. Testes de penetração, revisões de código e a implementação de práticas de segurança, como a validação de entradas e a atualização regular de componentes, são medidas importantes para mitigar essas vulnerabilidades, não permitindo que se tornem ameaças ou ataques concretizados, com consequências prejudiciais.

Proteção contra-ataques de injeção, cross-site scripting, entre outros

Proteger uma aplicação web contra ataques de injeção, cross-site scripting e outras vulnerabilidades é uma parte crítica da segurança da informação.

1. **Injeção (como SQL Injection).** Ao construir consultas SQL dinâmicas, utilize parâmetros preparados ou consultas parametrizadas. Isso evita a injeção de código SQL, pois os valores dos parâmetros são tratados separadamente do comando SQL. Pode-se fazer a validação e sanitização em todas as entradas do usuário. Certifique-se de que apenas dados válidos sejam aceitos e que qualquer entrada suspeita seja rejeitada ou tratada de forma segura. Nunca confie nas entradas do usuário, especialmente quando se trata de consultas de banco de dados; valide e trate sempre as entradas de forma defensiva.
2. **Cross-Site Scripting (XSS)** Escape HTML e JavaScript. Ao exibir dados fornecidos pelo usuário em páginas da web, escape todos os caracteres especiais de HTML e JavaScript, pois isso impede que scripts maliciosos sejam executados no navegador do usuário. Com o Content Security Policy (CSP), implemente uma política de segurança de conteúdo (CSP) que especifique de onde os recursos podem ser carregados. Isso ajuda a mitigar o risco de injeção de scripts.
3. **Cross-Site Request Forgery (CSRF).** Implemente tokens anti-CSRF para verificar a legitimidade das solicitações. Esses tokens são incluídos nas solicitações e verificados no servidor. Sempre exija autenticação para a execução de ações sensíveis, pois os invasores não podem explorar com sucesso o CSRF se não tiverem acesso às contas autenticadas.
4. Exija senhas fortes dos usuários, que incluam caracteres maiúsculos, minúsculos, números e símbolos, e implemente mecanismos de bloqueio de conta após várias tentativas de login mal sucedidas para impedir ataques de força bruta.
5. Ao permitir o upload de arquivos, imponha restrições rigorosas sobre tipos de arquivos, tamanho máximo e validação de conteúdo.
6. Garanta que as API exijam autenticação adequada e autorização para acessar recursos; para isso, use tokens de acesso e tokens de atualização.

Práticas seguras de desenvolvimento web

Práticas seguras de desenvolvimento web referem-se a diretrizes, técnicas e procedimentos que os desenvolvedores web seguem para criar aplicativos e sites online de forma a minimizar vulnerabilidades de segurança, proteger os dados dos usuários e garantir a integridade e a confiabilidade das aplicações. Essas práticas visam reduzir riscos relacionados a ataques cibernéticos, vazamento de informações e outras ameaças à segurança online. Entre as práticas, podemos exemplificar:

- A validação de entrada de dados visa garantir que os dados fornecidos pelos usuários sejam validados e sanitizados antes de serem processados para evitar ataques de injeção, como SQL Injection ou Cross-Site Scripting (XSS). Por exemplo, escapar caracteres especiais em campos de entrada para evitar injeção de scripts maliciosos.
- A autenticação e autorização que pretende implementar um sistema de autenticação robusto para garantir que apenas usuários autorizados tenham acesso a recursos sensíveis. Por exemplo, exigir senhas fortes e utilizar tokens de autenticação.

Outra ferramenta utilizada para a segurança de dados é o Data Loss Prevention, ou DLP, segundo Clarke (2022). Essa é uma ferramenta projetada para evitar a perda, o vazamento ou o uso não autorizado de dados sensíveis. Ela monitora o tráfego de rede, os dispositivos e os sistemas em busca de informações confidenciais, como números de cartão de crédito, informações pessoais ou segredos comerciais. O DLP aplica políticas e regras para prevenir a exfiltração de dados e tomar medidas corretivas quando necessário. O DLP consegue monitorar o tráfego de dados em tempo real, tanto na rede como nos endpoints em busca de atividades suspeitas ou violações de políticas de segurança. Ele analisa o conteúdo dos arquivos, os padrões de comunicação e os comportamentos dos usuários para encontrar possíveis vazamentos ou uso indevido de dados sensíveis. Apesar de proteger a rede, podemos pensar nesse recurso no contexto da web.

Utilizamos a criptografia para proteger dados confidenciais, tanto em repouso quanto em trânsito. Exemplos: criptografar senhas de usuários antes de armazená-las no banco de dados e usar HTTPS para proteger a comunicação entre o navegador e o servidor; fazer sempre as atualizações e patches para manter todas as bibliotecas, frameworks e servidores web atualizados com as últimas correções de segurança para evitar vulnerabilidades conhecidas.

Para proteção contra Cross-Site Request Forgery (CSRF) devemos implementar mecanismos para prevenir ataques CSRF, que podem ser usados para enganar os usuários a executar ações indesejadas em seu nome. Por exemplo, usar tokens anti-CSRF.

Temos que validar dados tanto no lado do servidor quanto no lado do cliente para evitar erros de validação. Por exemplo, se um usuário faz uma reserva em um site de reserva de hotéis, é importante verificar novamente no servidor se os dados são válidos, mesmo que a validação do lado do cliente já tenha sido realizada.